# BirdPath AI: Master Architecture, Concepts, Algorithms & Function Catalog

## Complete Technical Reference Document Across Python Deep Learning, Dual-Studio Streaming Server, and Android Kotlin Edge Systems

**Executive Overview:** The BirdPath platform is an end-to-end aerial computer vision, deep learning trajectory forecasting, and autonomous drone evasion system. It integrates real-time object detection (YOLOv8/YOLO11), multi-target tracking (ByteTrack with 8-state Kalman filtering), center-of-mass aerodynamic stabilization (Alpha-Beta filtering), deep sequence forecasting (2-layer GRU and Transformer Encoder models with heteroscedastic Gaussian uncertainty), 3D moving zone safety corridor projection, live GPU online backpropagation, and native Android edge execution.

## 1. High-Level Concepts & Engineering Domains

| Domain / Category | Core Principles & Methodology | Key Operational Topics & Components |
|---|---|---|
| **Deep Learning & Sequence Modeling** | Capturing non-linear temporal momentum, turn dynamics, and velocity deltas over multi-step temporal windows. | Recurrent Neural Networks (GRU), Transformer Encoders (Multi-Head Self-Attention), Sinusoidal Positional Encoding, Heteroscedastic Gaussian NLL Loss, Cosine Annealing LR. |
| **Computer Vision & Detection** | Spatial localization of flying birds under scale variations, background clutter, and high-speed motion blur. | YOLOv8/YOLO11 transfer learning, Tiled / SAHI 2x2 inference, Letterboxing with symmetric aspect padding, Greedy Non-Maximum Suppression (NMS), Temporal frame differencing. |
| **Multi-Object Tracking (MOT)** | Continuous identity maintenance and kinematic state estimation across occlusions and missed detections. | ByteTrack two-stage association (high/low score partitioning), 8-state linear Kalman filtering, Distance-gated Hungarian matching, Canonical track stitching, 30-frame ring buffering. |
| **Kinematics & Physics Decoupling** | Separating oscillating wing flapping from aerodynamic center-of-mass (CoM) trajectory. | Alpha-Beta filter (a=0.28, b=0.04), Constant Velocity (CV) & Constant Acceleration (CA) motion models, 85/15 Hybrid physics-residual blending, Centripetal banking, Thermal soaring. |
| **Camera Ego-Motion Compensation** | Distinguishing bird velocity from handheld phone rotation or FPV drone camera panning. | Lucas-Kanade pyramidal sparse optical flow, Shi-Tomasi corners, RANSAC partial affine estimation, Hardware 3-axis gyroscope sensor fusion via focal length projection. |
| **Real-Time Edge & Mobile Deployment** | Running compute-intensive deep vision and recurrence within constrained mobile power and latency budgets. | ONNX graph export, Float16 / INT8 quantization, Android CameraX (60 FPS buffer), TFLite GPU Delegate / NNAPI / CPU threading, Dynamic frame-skipping throttle. |
| **Robotics, Safety & FPV Integration** | Preventing mid-air bird strikes on autonomous drones and providing spatial awareness to FPV pilots. | 3D Cylindrical Moving Zone Tunnels, Catmull-Rom cubic splines, Drone evasion kinematics (90 deg orthogonal break vectors, TCPA), Stereoscopic Side-by-Side (SBS) VR HUD, Hardware pinouts. |

## 2. Mathematical Algorithms & Formulation Reference

| Algorithm Name | Mathematical State & Governing Equations | Implementation Purpose & Role |
|---|---|---|

| | | |
|---|---|---|
| **8-State Linear Kalman Filter** | `State: x = [cx, cy, w, h, vx, vy, vw, vh]^T`<br>`Predict: x_k\|k-1 = F*x_k-1, P_k\|k-1 = F*P*F^T + Q`<br>`Update: y = z - H*x, S = H*P*H^T + R, K = P*H^T*inv(S)`<br>`State Update: x = x + K*y, P = (I - K*H)*P` | Maintains continuous track position, smooths high-frequency noise, estimates velocity derivatives, and bridges occlusion gaps. |
| **ByteTrack Two-Stage Association** | `Cost Matrix: C(i, j) = 1 - IoU(T_i, D_j) if IoU >= 0.5 else 0.8 + 0.19*(dist/gate)`<br>`Stage 1: Hungarian assignment on high-conf detections (score >= 0.50)`<br>`Stage 2: Hungarian assignment on low-conf detections (0.10 <= score < 0.50)` | Prevents track fragmentation by preserving tracks during motion blur, scale transitions, and partial background occlusions. |
| **Center of Mass (CoM) Alpha-Beta Filter** | `Position: x_hat_k = x_pred + alpha * (z_k - x_pred), alpha = 0.28`<br>`Velocity: v_k = v_k-1 + (beta / dt) * (z_k - x_pred), beta = 0.04`<br>`Box Dimension EMA: s_k = 0.85 * s_k-1 + 0.15 * s_measured` | Decouples bounding box flapping oscillation from true kinetic trajectory, removing wing-beat noise from GRU training inputs. |
| **Gaussian NLL Loss with Smoothness Regularizer** | `L_NLL = 0.5 * sum[ log(sigma_k^2) + (y_k - mu_k)^2 / (sigma_k^2 + eps) ]`<br>`L_smooth = lambda_s * sum[ \|\| (mu_k+2 - mu_k+1) - (mu_k+1 - mu_k) \|\|^2 ]`<br>`Total Loss = L_NLL + L_smooth (lambda_s = 0.05, logvar in [-8.0, 4.0])` | Heteroscedastic loss learning both multi-horizon trajectory coordinates and expanding variance uncertainty while eliminating jerk. |
| **Hybrid Kinematics-Residual Blending** | `P_blended(k) = 0.85 * P_kinematic(k) + 0.15 * P_GRU(k)`<br>`where P_kinematic(k) = k*v_smooth + 0.5*(k^2)*a_clamped` | Ensures trajectory physically adheres to Newtonian mechanics while enabling the GRU to steer non-linear turns and dives. |
| **Catmull-Rom Cubic Spline Smoothing** | `P(t) = 0.5 * [ 2P1 + (-P0+P2)t + (2P0-5P1+4P2-P3)t^2 + (-P0+3P1-3P2+P3)t^3 ]`<br>`Continuous C1 curve with zero second-order acceleration discontinuities` | Smooths multi-step discrete waypoints into organic aerodynamic curves for rendering overlay tunnels and FPV HUDs. |
| **Drone Collision Evasion Kinematics** | `TCPA = - (r_rel . v_rel) / \|\|v_rel\|\|^2`<br>`Escape Vector: v_escape = v_bird x z_hat (90 deg orthogonal break)`<br>`Thrust Command: 85% - 95% burst throttle if TCPA < 1.8s and Dist < 150px` | Calculates autonomous collision avoidance waypoints for drones, ensuring immediate clearing of the 3D moving corridor. |

## 3. Python Deep Learning & Computer Vision Catalog (`python/`)

Contains all model definitions, training loops, synthetic generators, optical flow compensation, and export tools.

| Module / Script | Class / Function Name | Parameters & Signature | Role, Algorithm & Behavior |
|---|---|---|---|
| **models_trajectory.py** | `BirdTrajectoryGRU` | input_dim=6, hidden_dim=128, num_layers=2, future_steps=20, dropout=0.1 | Primary Trajectory Model: 2-layer GRU sequence encoder with MLP projection head. Output: (B, 20, 4) -> [mean_x, mean_y, logvar_x, logvar_y]. |
| **models_trajectory.py** | `BirdTrajectoryGRU.forward` | x: (Batch, 15, 6) | Runs GRU recurrence over 15 frames, takes final hidden state out[:, -1, :], passes through Linear(128,128)->ReLU->Linear(128,80). |
| **models_trajectory.py** | `PositionalEncoding` | d_model=128, max_len=50 | Vaswani sinusoidal positional encoding buffer generating interleaved sin/cos geometric progression across timesteps. |
| **models_trajectory.py** | `BirdTrajectoryTransformer` | input_dim=6, d_model=128, nhead=4, num_layers=2, future_steps=20 | Comparative Baseline Model: Multi-head self-attention TransformerEncoder (2 layers, 4 heads, feedforward=256) with identical MLP output head. |
| **models_trajectory.py** | `ConstantVelocityKalmanBaseline` | future_steps=20 | Kinematic baseline: extracts mean velocity over last 5 frames (dx, dy) and linearly extrapolates with step multipliers 1..20. |
| **models_trajectory.py** | `ConstantAccelerationBaseline` | future_steps=20 | Kinematic baseline: estimates velocity and acceleration a = (v_last - v_prev)/4; predicts via s = vt + 0.5*a*t^2. |
| **models_trajectory.py** | `GaussianNLLWithSmoothness` | lambda_smoothness=0.05 | Loss function combining heteroscedastic Gaussian Negative Log-Likelihood with 2nd-order finite difference trajectory acceleration penalty. |
| **models_trajectory.py** | `HuberWithSmoothness` | delta=0.01, lambda_smoothness=0.05 | Alternative robust loss using Huber loss on coordinate offsets plus trajectory smoothness penalty. |
| **train_trajectory.py** | `compute_metrics` | pred_mean: (N, 20, 2), target: (N, 20, 2) | Computes Average Displacement Error (ADE) and Final Displacement Error (FDE) at step horizons 5, 10, and 20. |
| **train_trajectory.py** | `train_model` | model_type='gru', data_path, epochs=40, batch=64, lr=1e-3, save_dir | Supervised training loop: loads .npz data, executes AdamW with Cosine Annealing, L2 gradient clipping (1.0), saves best checkpoint. |
| **evaluate_trajectory.py** | `evaluate_all` | data_path, gru_ckpt, transformer_ckpt | Multi-model benchmark: evaluates CV Kalman, CA, GRU, and Transformer across Straight, Turning, and Soaring flight subsets. |
| **dataset_builder.py** | `SyntheticBirdTrajectoryGenerator` | frame_w=1920, frame_h=1080, fps=30.0 | Physics flight simulator synthesizing straight glides, coordinated banking turns, and thermal soaring paths with Brownian noise. |
| **dataset_builder.py** | `extract_windows_from_track` | track_boxes, history_len=15, future_len=20 | Sliding window feature extractor converting box arrays to history tensor X (N, 15, 6) and future offset target Y (N, 20, 2). |
| **dataset_builder.py** | `build_synthetic_dataset` | num_tracks_per_type=500, save_path | Generates full training dataset across flight styles with 70/15/15 train/val/test splits saved to .npz format. |
| **dataset_builder.py** | `parse_phone_recorded_json` | json_path | Parses real-world flight tracks logged by the Android app into NumPy trajectory sequences (filtering tracks >= 35 frames). |
| **tracker.py** | `KalmanBoxTracker` | bbox: [cx, cy, w, h], score, dt=1.0 | 8-state bounding box Kalman Filter with constant velocity transition model, covariance tracking, and 30-frame history ring buffer. |
| **tracker.py** | `ByteTracker` | high_thresh=0.5, low_thresh=0.1, match_thresh=0.5, max_time_lost=30 | ByteTrack two-stage association engine with distance-gated cost matrix and Hungarian matching (scipy linear_sum_assignment). |
| **tracker.py** | `bbox_iou` | box1, box2 | Calculates geometric Intersection-over-Union (IoU) between two [cx, cy, w, h] bounding boxes. |
| **camera_compensation.py** | `CameraMotionCompensator` | max_corners=200, quality_level=0.01, min_distance=15.0 | Camera ego-motion estimator: tracks Shi-Tomasi corners on masked background with Lucas-Kanade optical flow, solves RANSAC affine matrix. |

| | | | |
|---|---|---|---|
| **camera_compensation.py** | `warp_points` | points: (N, 2), M: (2, 3) | Static method: warps historical trajectory points using inverse affine transformation to align with current camera coordinates. |
| **motion_hint.py** | `MotionHintExtractor` | diff_threshold=25, downsample_w=320, downsample_h=180, boost=0.20 | Temporal frame difference engine: detects moving pixels with morphological opening/dilation and boosts small bird detection scores. |
| **train_detector.py** | `train_yolo` | data_yaml, model_name='yolov8n.pt', epochs=50, imgsz=640, batch=16 | Fine-tunes YOLOv8n single-class bird detector with mosaic (1.0), scale (0.5), HSV, and perspective augmentations. |
| **train_detector.py** | `TiledInference` | detector_func, crop_size=640, overlap=0.2 | Slices high-resolution video into 2x2 overlapping quadrant tiles for small bird detection and merges results with greedy NMS. |
| **export_tflite.py** | `export_torch_to_onnx` | model, onnx_path='bird_trajectory.onnx' | Exports PyTorch BirdTrajectoryGRU to ONNX (opset 14) with dynamic batch dimensions and constant folding. |
| **export_tflite.py** | `convert_onnx_to_tflite` | onnx_path, tflite_f16_path, tflite_int8_path, calib_data | Converts ONNX graph to Float16 and INT8 quantized TFLite models using representative calibration datasets. |
| **export_tflite.py** | `verify_numeric_parity` | torch_model, tflite_path, tolerance=1e-3 | Numerical parity checker: confirms that PyTorch and TFLite inference outputs match within 1e-3 tolerance. |

## 4. Dual-Studio Server & Online Learning Catalog (`computer learning/`)

Manages real-time video stream ingestion, dual-studio sessions, online GPU backpropagation, and visual rendering.

| Module / Script | Class / Method Name | Parameters & Signature | Role, Algorithm & Behavior |
|---|---|---|---|
| online_trainer.py | `OnlineTrajectoryTrainer` | checkpoint_path, learning_rate=1e-4, device | Online learning coordinator: loads PyTorch BirdTrajectoryGRU, AdamW optimizer, and maintains loss/ADE/FDE telemetry. |
| online_trainer.py | `OnlineTrajectoryTrainer.predict` | x_history: (15, 6) or (B, 15, 6) | Forward pass inference: returns 20 future predicted offset means and log-variance uncertainty bounds. |
| online_trainer.py | `OnlineTrajectoryTrainer.train_step` | x_samples, y_actual_samples | Live GPU backpropagation: computes Gaussian NLL loss, clips gradients (<= 1.0), executes AdamW step, records telemetry. |
| online_trainer.py | `OnlineTrajectoryTrainer.save_checkpoint` | path, split_name | Persists model weights, optimizer states, and sample counters; mirrors checkpoints to root storage directory. |
| tracker_engine.py | `VideoTrackerEngine` | trainer, yolo_model_path='yolov8n.pt', conf=0.25 | Master per-frame tracking pipeline coordinator: integrates YOLOv8, ByteTrack, CoM stabilizer, and training harvest. |
| tracker_engine.py | `VideoTrackerEngine.process_frame` | frame_bgr: np.ndarray | Executes full pipeline: YOLO detection -> ByteTrack -> Track stitching -> CoM Alpha-Beta filter -> 85/15 hybrid forecast -> train harvest. |
| tracker_engine.py | `CoM Alpha-Beta Filter` | alpha=0.28, beta=0.04 (internal) | Suppresses wing-flapping bounding box noise to extract steady aerodynamic center-of-mass velocity and position. |
| tracker_engine.py | `Track Stitching Engine` | max_dist=160px, max_gap=90f (internal) | Re-identifies birds that temporarily disappear or cross occlusions, restoring historical timelines and canonical track IDs. |
| visualizer.py | `LayerVisualizer` | None | 7-layer modular rendering engine: manages toggles for video, boxes, actual paths, forecasts, tunnels, rings, and HUD graphs. |
| visualizer.py | `LayerVisualizer.render` | frame_bgr, birds, horizon=20 | Composites all active visual layers onto frame canvas: color-coded bounding boxes, splines, and moving zone corridors. |
| visualizer.py | `LayerVisualizer._draw_tunnel` | canvas, pred_pts, radii, color, fps, horizon | Renders 3D moving cylinder tunnel: translucent glow envelope + circular rib rings at 1s, 2s, 3s, and 4s arrival milestones. |
| visualizer.py | `LayerVisualizer._draw_inset_graph` | canvas, pred_pts, actual_pts, fps, horizon | Renders a 220x110 px live telemetry inset HUD mini-graph plotting X and Y coordinates vs time. |
| visualizer.py | `LayerVisualizer._draw_drone_vr_hud` | canvas, birds, fps | Renders UAV FPV cockpit HUD: artificial horizon, pitch ladder (+/-10 deg), compass tape, target lock, and evasion vector. |
| visualizer.py | `LayerVisualizer._make_sbs_vr` | canvas, ipd=14 | Transforms composite frame into stereoscopic Side-by-Side (SBS) dual-eye VR format with horizontal disparity and lens vignette. |
| youtube_loader.py | `YouTubeStreamLoader` | download_dir='downloads' | Video stream resolution engine: checks local cache, validates YouTube URLs, downloads via yt-dlp, and verifies format integrity. |
| server.py | `StudioSession` | name, is_testing, trainer | Encapsulates state for isolated learning or testing workspaces: manages video capture, tracker engine, FPS, and telemetry. |
| server.py | `StudioSession._run_loop` | None (async worker) | Main frame loop: reads OpenCV VideoCapture, calls tracker_engine.process_frame(), visualizer.render(), and updates JPEG buffer. |
| server.py | `StudioSession.generate_mjpeg` | None (async generator) | Streams multipart JPEG frames over HTTP at sustained 30 FPS with boundary headers. |
| server.py | `StudioSession.get_drone_evasion_telemetry` | None | Calculates TCPA, closure velocity, threat severity, and generates simulated MAVLink v2.0 collision avoidance commands. |
| server.py | `StudioSession.render_bird_plot` | bird_id: int | Generates high-resolution Matplotlib trajectory comparison plot (Gaussian-smoothed actual vs predicted with milestone markers). |

| | | | |
|---|---|---|---|
| **server.py** | `FastAPI Endpoints` | 40+ REST / WebSocket routes | Provides REST control interface: start/pause/stop sessions, toggle layers 1-7, select/save checkpoints, and poll telemetry. |

## 5. Native Android Mobile Application Catalog (`android/`)

Native Kotlin implementation running real-time detection, tracking, prediction, and overlays on mobile devices.

| Kotlin File / Component | Class / Method Name | Parameters & Signature | Role, Algorithm & Behavior |
|---|---|---|---|
| **MainActivity.kt** | `MainActivity` | AppCompatActivity() | Master Android activity: orchestrates CameraX preview/analysis, permission lifecycle, background threads, and UI widgets. |
| **MainActivity.kt** | `processCameraFrame` | imageProxy: ImageProxy | Core per-frame Android pipeline: applies gyro compensation, executes YOLO inference, ByteTrack association, and triggers OverlayView. |
| **MainActivity.kt** | `startCamera` | None | Initializes CameraX Preview and ImageAnalysis (1080p, 60 FPS, STRATEGY_KEEP_ONLY_LATEST, RGBA_8888) with back-facing camera. |
| **YoloDetector.kt** | `YoloDetector` | context, modelFileName='yolov8n_bird.tflite' | TFLite single-class bird detector with hardware fallback: tries GPU Delegate -> NNAPI -> CPU (4 threads). |
| **YoloDetector.kt** | `runSingleInference` | bitmap, confThreshold, iouThreshold | Preprocesses bitmap via 640x640 letterboxing, executes TFLite inference over 8,400 anchors, and applies greedy NMS. |
| **YoloDetector.kt** | `runTiledInference` | bitmap, confThreshold, iouThreshold | Splits input into 2x2 overlapping quadrant tiles (15% overlap), runs inference per tile, translates coordinates, and merges. |
| **KalmanFilter.kt** | `KalmanFilter` | initBox: FloatArray [cx, cy, w, h], dt=1.0f | Native Kotlin 8-state Kalman Filter: maintains state [cx, cy, w, h, vx, vy, vw, vh] and covariance P with constant velocity model. |
| **KalmanFilter.kt** | `invert4x4` | m: Array (companion) | Analytical 4x4 matrix inversion using cofactor expansion and determinant for fast, garbage-free Kalman gain updates. |
| **ByteTracker.kt** | `ByteTracker` | highThresh=0.50f, lowThresh=0.10f, matchThresh=0.40f | Mobile two-stage association tracker: tracks lifecycles (TENTATIVE -> CONFIRMED -> LOST -> REMOVED) using IoU bipartite matching. |
| **ByteTracker.kt** | `minCostMatching` | cost: Array, maxCost: Float | Greedy minimum-cost assignment algorithm: sorts candidate cost pairs and greedily assigns matches for mobile real-time performance. |
| **TrajectoryPredictor.kt** | `TrajectoryPredictor` | context, modelFileName='bird_trajectory_fp16.tflite' | TFLite Float16 2-layer GRU inference engine: maps 15 past tracking frames to 20 future coordinate waypoints and uncertainty radii. |
| **TrajectoryPredictor.kt** | `predictWithKalmanVelocity` | track, horizon=20 | Kinematic fallback predictor: performs linear extrapolation using Kalman velocity when track history contains fewer than 5 frames. |
| **CameraMotionCompensator.kt** | `CameraMotionCompensator` | context: Context | Hardware gyroscope listener (SENSOR_DELAY_FASTEST): converts angular rates (omegaX, omegaY) to pixel shifts via focal geometry. |
| **CameraMotionCompensator.kt** | `consumeShift` | None -> Pair | Returns and resets accumulated pixel displacement (dx, dy) to subtract phone pan/tilt motion from Kalman track states. |
| **FrameSkipController.kt** | `FrameSkipController` | targetFps=20.0f | Dynamic throttle: skips heavy YOLO detection when processing exceeds latency budget (50 ms), maintaining continuous 60 Hz Kalman updates. |
| **CoordinateMapper.kt** | `CoordinateMapper` | frameW, frameH, modelW=640, modelH=640, viewW, viewH | Geometry transformation engine: handles un-letterboxing, sensor orientation rotation (0/90/180/270 deg), and CameraX FILL_CENTER scaling. |
| **CatmullRomSpline.kt** | `CatmullRomSpline` | Object (singleton) | C1-continuous cubic spline generator: interpolates discrete predicted waypoints into smooth curves with 6 subdivisions per segment. |
| **OverlayView.kt** | `OverlayView` | View(context, attrs) | Hardware-accelerated custom view: renders Layer 2 bounding boxes and Layer 3 trajectory curves with fading alpha and 500 ms lost-track fade. |
| **TrackRecorder.kt** | `TrackRecorder` | context: Context | Active learning logger: serializes timestamped bounding boxes and track histories to structured JSON on local storage. |

| | | | |
|---|---|---|---|
| **SettingsManager.kt** | `SettingsManager` | context: Context | SharedPreferences manager: persists confidence thresholds, prediction horizons, tiling flags, and layer toggle preferences. |

## 6. Verification Suites, Tests & Media Generators

Automated unit tests, publication documents, presentation generators, and system integration diagrams.

| File / Module | Function / Class Name | Scope & Parameters | Verification Purpose & Output |
|---|---|---|---|
| test_tracker.py | `test_kalman_filter_predict_and_update` | Unit test (pytest) | Verifies Kalman filter state advancement, hit counter increments, and positive velocity updates upon target motion. |
| test_tracker.py | `test_bytetrack_association_two_stages` | Unit test (pytest) | Tests ByteTrack two-stage association across high and low detection scores and confirms promotion to CONFIRMED after 3 hits. |
| test_tracker.py | `test_track_lifecycle_removal_after_30_misses` | Unit test (pytest) | Confirms that tracks transition to LOST and are evicted on the 31st consecutive miss (max_time_lost=30). |
| test_tracker.py | `test_ring_buffer_maintains_30_positions` | Unit test (pytest) | Validates that KalmanBoxTracker's internal deque strictly caps historical entries at 30 samples after 50 update cycles. |
| test_trajectory_preprocessing.py | `test_trajectory_window_dimensions` | Unit test (pytest) | Verifies that extract_windows_from_track yields tensor shapes X (N, 15, 6) and Y (N, 20, 2) on a 50-frame trajectory. |
| test_trajectory_preprocessing.py | `test_deltas_correctness` | Unit test (pytest) | Checks that dx and dy columns accurately represent frame-to-frame velocity differences and cumulative offset targets. |
| test_trajectory_preprocessing.py | `test_short_track_rejection` | Unit test (pytest) | Ensures trajectories with fewer than 35 frames (15 history + 20 future) are safely rejected with empty output arrays. |
| test_coordinate_mapping.py | `letterbox_box & unletterbox_box` | Unit test (pytest) | Verifies bidirectional coordinate transformation between original 1080p frame and 640x640 padded model space. |
| test_coordinate_mapping.py | `test_screen_fill_center_mapping` | Unit test (pytest) | Validates CameraX FILL_CENTER aspect ratio scaling, ensuring frame center maps precisely to screen center on portrait displays. |
| build_full_ppt.py | `generate_master_presentation` | Output: BirdPath_Full_Project_Master.pptx | Generates comprehensive 21-slide master PowerPoint covering full architecture, dual studios, datasets, and flight tunnels. |
| build_full_parameters_pdf.py | `build_parameters_pdf` | Output: BirdPath_Full_System_Parameters_Master.pdf | Generates publication-grade PDF documenting all system parameters, exact values, usage frequencies, and hardware pinouts. |
| generate_ppt.py | `create_presentation` | Output: BirdPath_System_Explanation.pptx | Generates 11-slide visual guide explaining vision, tracking, GRU prediction, online backprop, and telemetry dashboards. |
| generate_simple_pipeline_picture.py | `generate_clean_pipeline_image` | Output: drone_to_engine_to_vr.png | Generates 2400x950 px high-res graphic depicting FPV Drone -> Ground Dock [V G P G G 5 T R] -> AI Engine -> VR Goggles. |
| generate_system_diagram_image.py | `create_system_diagram` | Output: hardware_and_machine_integration_diagram.png | Generates 2560x1600 px master architecture diagram with pinout cards, dual-studio screens, and evasion HUD telemetry. |